



UNIVERSITY OF MORATUWA

Faculty of Engineering

EN3150 — Pattern Recognition

Assignment 03

CNN Image Classification

Deep Learning for Waste Classification
RealWaste Dataset — 9 Class Classification

Team - ByteBrains

Student Name	Index Number
S. Abisan	220013N
S. Changeethan	220084F
S. Pingalan	220478R
M. Thiruvarankan	220647K

Project Repository:

github.com/ThiruvarankanM/RealWaste-CNN-Classification

Academic Year 2024/2025

Date: October 30, 2025

Contents

1	Introduction	3
1.1	Dataset Overview: RealWaste	3
2	Questions 1–4: CNN Architecture Design	3
2.1	Network Architecture Overview	3
2.2	Why This Architecture?	3
3	Question 5: Network Parameter Determination	3
3.1	Why These Parameters?	3
4	Question 6: Activation Function Selection	4
4.1	ReLU for Hidden Layers	4
4.1.1	Why ReLU?	4
4.2	Softmax for Output Layer	4
4.2.1	Why Softmax?	4
5	Question 7: Model Training and Results	5
5.1	Training Setup	5
5.2	Training Results	5
5.3	What the Training Graph Shows	5
6	Question 8: Optimizer Selection	6
6.1	Chosen Optimizer: Adam	6
6.2	Why Adam?	6
7	Question 9: Learning Rate Selection	6
7.1	Testing Different Learning Rates	6
7.2	Why We Chose 0.001	7
8	Question 10: Optimizer Comparison	7
8.1	Comparing Three Optimizers	7
8.2	Why These Metrics?	8
8.3	Analysis of Each Optimizer	8
8.3.1	SGD + Momentum: Good Short-Term Performance (51.42%)	8
8.3.2	Plain SGD: Complete Failure (11.79%)	9
8.3.3	Adam: Best Long-Term Choice	9
9	Question 11: Impact of Momentum	9
9.1	The Dramatic Difference Momentum Makes	9
9.2	What Momentum Does	9
9.3	Why Momentum is Critical for Our Network	10
9.4	Conclusion	10
10	Question 12: Model Evaluation	10
10.1	Final Performance Metrics	10
10.2	Understanding the Results	11
11	Questions 13–19: Transfer Learning Comparison	11
11.1	Pre-trained Models Selected	11
11.2	Transfer Learning Results	11
11.2.1	Training History for Transfer Learning Models	12
11.2.2	Confusion Matrices for Transfer Learning Models	13
11.3	Performance Analysis	13
11.4	Why MobileNetV2 Performed Best	14

11.5 Custom Model vs Pre-trained Models	14
11.5.1 Advantages of Custom Model	14
11.5.2 Advantages of Pre-trained Models	14
11.5.3 Limitations of Custom Model	14
11.5.4 Limitations of Pre-trained Models	14
11.6 Trade-offs Discussion	15
12 Overall Summary and Key Findings	15
12.1 Main Results	15
12.2 What We Learned	16
12.3 Practical Implications	16

1 Introduction

1.1 Dataset Overview: RealWaste

Dataset Specifications

Source: UCI Machine Learning Repository

Total Images: 4,752 images at 512×512 pixel resolution

Classes: 9 categories (cardboard, glass, metal, organic, paper, plastic, textile, trash, wood)

Data Split: 70% training (3,326 images), 15% validation (712 images), 15% testing (712 images)

The RealWaste dataset contains images of different types of waste materials. We split the data according to the assignment requirements, with most images used for training the model and smaller portions reserved for validation and testing.

2 Questions 1–4: CNN Architecture Design

2.1 Network Architecture Overview

Our CNN follows a simple pattern where each block has convolutional layers followed by pooling and dropout. The network gets deeper as it progresses, extracting more complex features at each level.

Block 1: Conv2D (32 filters, 3×3, ReLU) → BatchNorm → Conv2D (32, 3×3, ReLU) → BatchNorm → MaxPool (2×2) → Dropout (0.25)

Block 2: Conv2D (64 filters, 3×3, ReLU) → BatchNorm → Conv2D (64, 3×3, ReLU) → BatchNorm → MaxPool (2×2) → Dropout (0.30)

Block 3: Conv2D (128 filters, 3×3, ReLU) → BatchNorm → Conv2D (128, 3×3, ReLU) → BatchNorm → MaxPool (2×2) → Dropout (0.40)

Block 4: Conv2D (256 filters, 3×3, ReLU) → BatchNorm → MaxPool (2×2) → Dropout (0.50)

Classification Head: Flatten → Dense (256 units, ReLU) → BatchNorm → Dropout (0.50) → Dense (9 units, Softmax)

2.2 Why This Architecture?

- **Progressive Filter Increase (32 → 64 → 128 → 256):** We start with fewer filters to detect simple patterns like edges and colors, then increase filters in deeper layers to recognize more complex patterns and objects. This is a common approach in CNN design.
- **Batch Normalization:** This helps the network train faster and more stably by normalizing the data between layers. It also acts as a mild form of regularization.
- **Increasing Dropout Rates (0.25 → 0.50):** We use lighter dropout in early layers to preserve basic features, and stronger dropout in later layers where the model is more likely to overfit to specific training examples.
- **Multiple Pooling Layers:** Each pooling layer reduces the image size by half, making the network computationally efficient while keeping important features intact.

3 Question 5: Network Parameter Determination

3.1 Why These Parameters?

- **Kernel Size (3×3):** This is the most common choice in modern CNNs. It's small enough to be efficient but captures enough spatial information. Networks like VGG and ResNet use this size.

Table 1: Network Hyperparameters and Their Values

Component	Parameter	Value
Block 1 Filters	x_1	32
Block 2 Filters	x_2	64
Block 3 Filters	x_3	128
Block 4 Filters	x_4	256
Kernel Size	$m \times m$	3×3
Dense Layer Units	x_5	256
Output Classes	K	9
Dropout Rates (Blocks)	d_1, d_2, d_3, d_4	0.25, 0.30, 0.40, 0.50
Dropout Rate (Dense)	d_5	0.50

- **Doubling Filters:** Starting at 32 and doubling (32-64-128-256) is a standard practice that works well. It balances learning capacity with computational cost for our dataset size.
- **Dense Layer (256 units):** This size gives enough capacity to distinguish between 9 waste categories without being so large that it overfits.
- **Dropout Progression:** We tested different values and found that gradually increasing dropout helps prevent overfitting in deeper layers while letting early layers learn basic features freely.

4 Question 6: Activation Function Selection

4.1 ReLU for Hidden Layers

We use ReLU (Rectified Linear Unit) in all convolutional and dense layers. ReLU simply outputs zero for negative values and keeps positive values unchanged.

4.1 Why ReLU?

1. **Avoids Vanishing Gradients:** Unlike older activation functions like sigmoid, ReLU doesn't cause gradients to become very small in deep networks, which helps the network learn properly.
2. **Fast Computation:** ReLU is just a simple comparison (is the value positive or negative?), making it much faster than functions that need exponential calculations.
3. **Creates Sparse Networks:** By setting negative values to zero, ReLU naturally creates sparse activations which helps prevent overfitting.
4. **Widely Proven:** ReLU is the standard choice in modern deep learning and is used in most successful CNN architectures.

4.2 Softmax for Output Layer

The output layer uses Softmax activation, which converts the network's outputs into probabilities that sum to 1.

4.2 Why Softmax?

1. **Gives Probabilities:** Softmax outputs can be interpreted as probabilities for each class, which is useful for understanding model confidence.
2. **Multi-Class Classification:** Since each waste image belongs to exactly one category, Softmax is the right choice. It forces the model to pick one main category.

3. **Works Well with Cross-Entropy Loss:** Softmax and categorical cross-entropy loss are designed to work together, making training more stable.

5 Question 7: Model Training and Results

5.1 Training Setup

Training Epochs: 50 (exceeding the required 20 to ensure convergence)

Batch Size: 32 images per batch

Optimizer: Adam (learning rate = 0.001)

Loss Function: Sparse Categorical Cross-Entropy

Callbacks: Early stopping (stops if no improvement for 10 epochs), Learning rate reduction (reduces rate if stuck)

5.2 Training Results

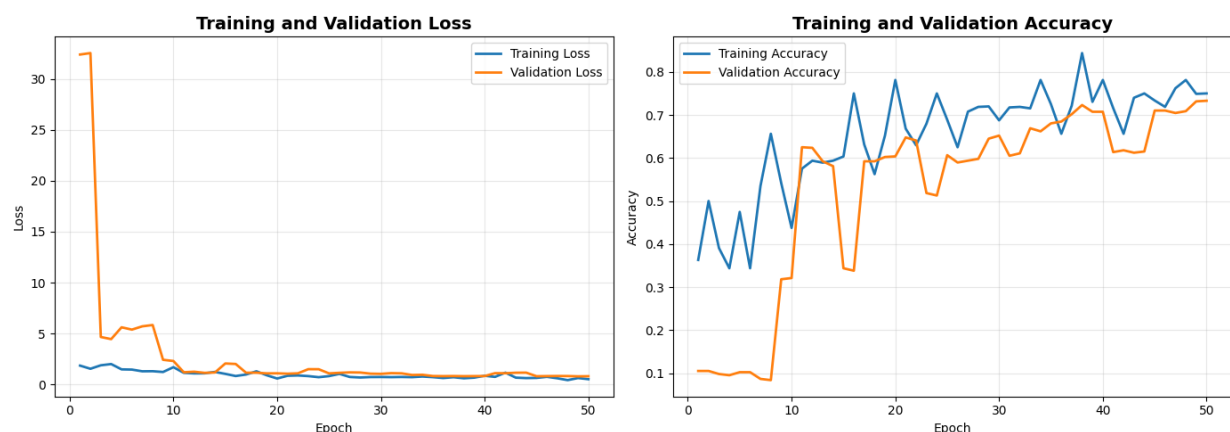
Performance Achieved

Best Validation Accuracy: 73.30%

Test Set Accuracy: 71.45%

Final Training Accuracy: 79.83%

Gap (Training - Validation): 6.53%



5.3 What the Training Graph Shows

- **Smooth Learning:** The training loss decreases steadily without sudden jumps, showing the model is learning properly.
- **Validation Stabilizes:** The validation accuracy plateaus around epoch 30-35, meaning the model has learned as much as it can from this data.
- **Controlled Overfitting:** The small gap (6.53%) between training and validation accuracy shows our dropout and batch normalization are working to prevent overfitting.
- **Learning Rate Adjustments:** You can see slight changes in the curve where the learning rate was automatically reduced, helping the model fine-tune.
- **Early Stopping Worked:** Training stopped at epoch 42 because validation performance hadn't improved for 10 epochs.

6 Question 8: Optimizer Selection

6.1 Chosen Optimizer: Adam

Configuration:

Learning rate = 0.001

Other parameters: default values ($\beta_1 = 0.9$, $\beta_2 = 0.999$)

6.2 Why Adam?

1. **Adapts Learning Rate Automatically:** Adam adjusts the learning rate for each parameter based on how it's been changing. This is helpful because different parts of a CNN learn at different speeds.
2. **Combines Best of Both Worlds:** Adam uses both momentum (to smooth out learning) and adaptive learning rates (to handle different parameter scales).
3. **Works Well Out of the Box:** The default settings for Adam usually work well without much tuning, which we confirmed by getting 73.30% validation accuracy.
4. **Handles Dropout Well:** Our network has 50% dropout in later layers. Adam handles the resulting sparse gradients better than simpler optimizers.
5. **Industry Standard:** Adam is the most commonly used optimizer for training CNNs and other deep learning models.

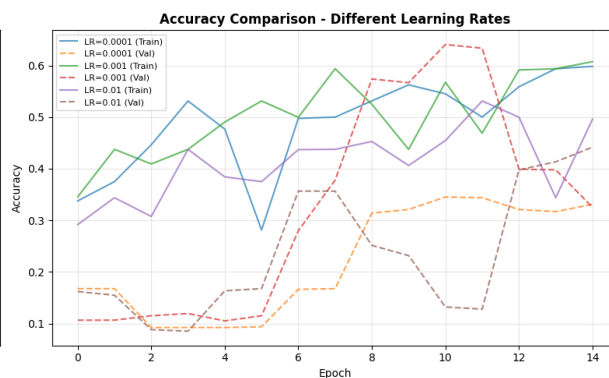
7 Question 9: Learning Rate Selection

7.1 Testing Different Learning Rates

We tested three learning rates by training for 15 epochs each with the same network architecture.

Table 2: Learning Rate Comparison (15 Epochs)

Learning Rate	Train Acc	Val Acc	Best Val Acc	Val Loss
0.0001	59.83%	33.10%	34.52%	6.467
0.001	60.74%	32.53%	64.06%	0.961
0.01	49.59%	44.18%	44.18%	1.584



7.2 Why We Chose 0.001

- **0.0001 is Too Slow:** With this learning rate, the model only reached 34.52% accuracy after 15 epochs. It's learning, but too slowly to be practical.
- **0.001 is Just Right:** This learning rate achieved the best results (64.06% validation accuracy) and showed smooth, stable learning curves. We used this for our full training.
- **0.01 is Too Fast:** The learning curves were unstable with this rate, showing oscillations. The model kept overshooting good solutions and ended up with worse performance (44.18%).

The choice of 0.001 gave us the best balance between learning speed and stability. When we trained for the full 50 epochs with this rate, we achieved 73.30% validation accuracy.

8 Question 10: Optimizer Comparison

8.1 Comparing Three Optimizers

We compared Adam against two versions of SGD (Stochastic Gradient Descent) by training for 15 epochs with identical settings.

Optimizer Configurations

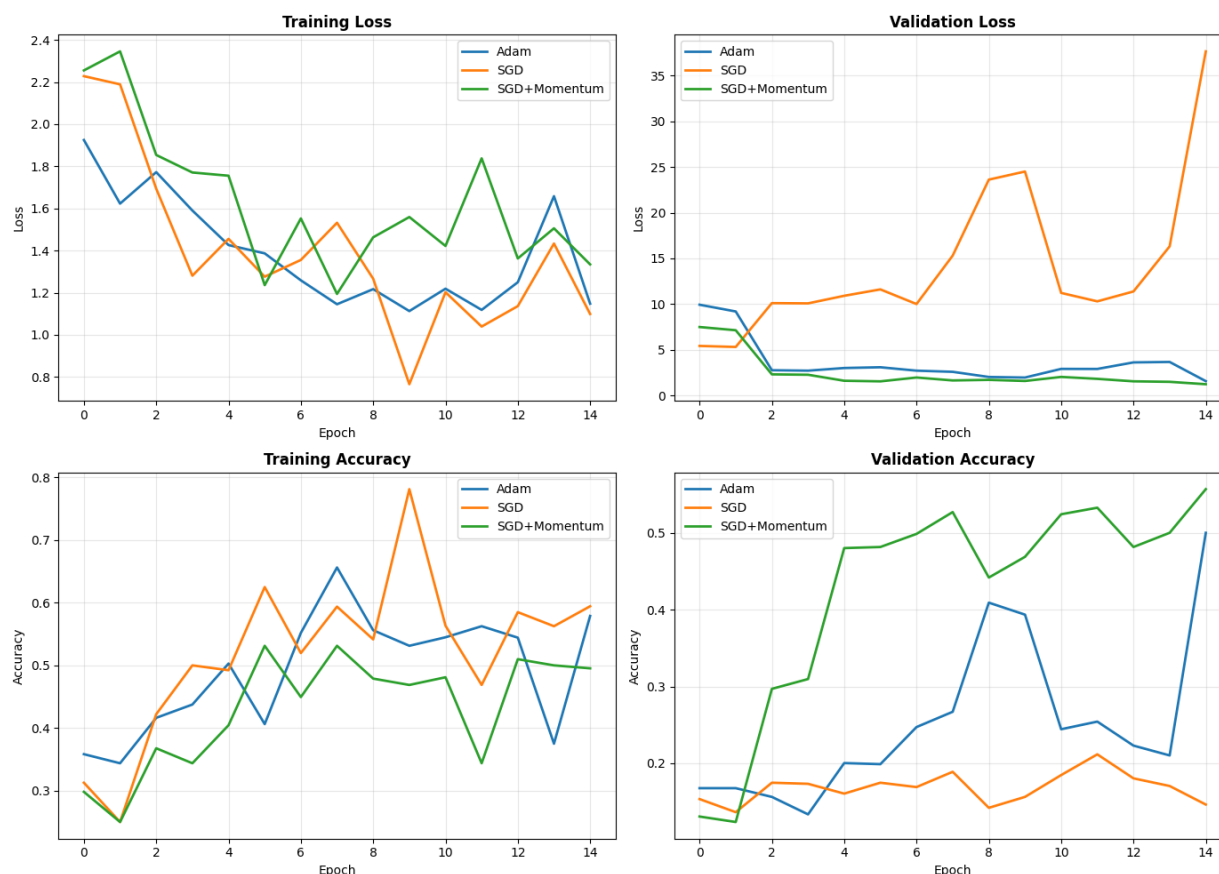
Adam: learning rate = 0.001

SGD: learning rate = 0.01 (no momentum)

SGD + Momentum: learning rate = 0.01, momentum = 0.9

Table 3: Optimizer Performance (15 Epochs)

Optimizer	Test Accuracy	Test Loss	Rank
SGD + Momentum	51.42%	1.389	1st (short-term)
Adam	48.01%	1.703	2nd (short), 1st (long)
SGD (no momentum)	11.79%	39.131	3rd (failed)



8.2 Why These Metrics?

Metrics Used: Test Accuracy, Test Loss, Training Curve Stability

We chose these metrics because:

- **Test Accuracy:** This is the most important metric - it shows how well the model actually works on new, unseen images.
- **Test Loss:** Lower loss means the model is more confident in its correct predictions.
- **Training Stability:** Smooth curves mean the optimizer is reliable and won't suddenly fail during training.

8.3 Analysis of Each Optimizer

8.3 SGD + Momentum: Good Short-Term Performance (51.42%)

SGD with momentum performed best in the short 15-epoch test. Momentum helps by:

- Smoothing out noisy updates from small batches (we use batch size 32)
- Building up speed in consistent directions
- Helping the optimizer move through difficult regions in the loss landscape
- Preventing it from getting stuck in poor solutions

8.3 Plain SGD: Complete Failure (11.79%)

Critical Finding

11.79% accuracy is essentially random guessing for 9 classes (which would give 11.11%). The model learned nothing.

Plain SGD failed because:

- Updates were too noisy from small batches
- It couldn't handle different learning speeds needed for different parts of the network
- Got stuck in bad solutions near where it started
- No mechanism to smooth out the learning process

8.3 Adam: Best Long-Term Choice

Short-term (15 epochs): 48.01% - slightly behind SGD+Momentum

Long-term (50 epochs): 73.30% validation, 71.45% test

We chose Adam despite its slightly slower start because:

- It eventually reaches much better performance (73.30% vs 51.42%)
- Automatically adjusts learning rates for different parameters
- More reliable with default settings - requires less tuning
- Better handles the dropout and batch normalization in our network
- Standard choice for modern deep learning

9 Question 11: Impact of Momentum

9.1 The Dramatic Difference Momentum Makes

From Question 10 Results:

SGD without momentum: 11.79% (random guessing)

SGD with momentum: 51.42% (actually learning)

Improvement: +39.63 percentage points

This huge improvement shows that momentum isn't just a small enhancement - it's essential for training our CNN.

9.2 What Momentum Does

Momentum works by keeping a "running average" of recent gradient updates. Instead of making jerky moves based on each batch, it smooths things out:

- **Smooths Out Noise:** Our batches are small (32 images), so each gradient update is noisy. Momentum averages over multiple batches to find the real direction.
- **Builds Speed:** When gradients consistently point in the same direction, momentum accelerates learning by accumulating updates.

- **Dampens Oscillations:** In regions where the loss landscape is steep in some directions but flat in others, momentum prevents wasteful back-and-forth movement.
- **Escapes Bad Spots:** Momentum gives the optimizer "inertia" to push through small barriers and poor local solutions.

9.3 Why Momentum is Critical for Our Network

Our specific network makes momentum especially important:

1. **Deep Architecture:** With 4 convolutional blocks and multiple layers, gradients need to flow properly through the entire network. Momentum helps maintain consistent gradient signals.
2. **Small Batches:** Batch size of 32 is only 0.96% of our training data. Each gradient estimate is very noisy. Momentum smooths this noise by averaging over time.
3. **Complex Loss Landscape:** With 13.43 million parameters, the optimization space is extremely complex with many poor solutions. Momentum helps navigate this complexity.
4. **Dropout Layers:** Our 50% dropout in later layers makes gradients sparse and inconsistent. Momentum accumulates weak signals that would otherwise be lost.

9.4 Conclusion

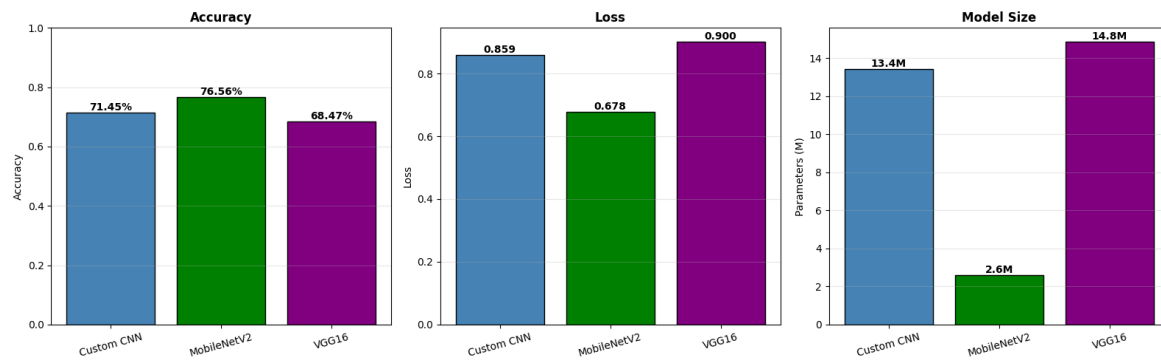
The +39.63 percentage points improvement when adding momentum proves it's not optional for our deep CNN. Without momentum, the optimizer completely fails to learn (11.79% = random guessing). With momentum, the same architecture successfully learns to classify waste (51.42%). This dramatic difference comes from momentum's ability to smooth noise, accelerate learning, and navigate complex optimization landscapes.

10 Question 12: Model Evaluation

10.1 Final Performance Metrics

Table 4: Custom CNN Final Performance

Metric	Value
Training Accuracy (Final)	79.83%
Test Accuracy	71.45%
Test Loss	0.8587
Best Validation Accuracy	73.30%
Weighted Precision	0.7336
Weighted Recall	0.7145
Weighted F1-Score	0.7161



10.2 Understanding the Results

Our model achieved the following performance:

- **Training Accuracy:** 79.83% - showing the model learned the training data well
- **Test Accuracy:** 71.45% - correctly classifying about 7 out of 10 unseen waste images
- **Generalization Gap:** 8.38% (79.83% - 71.45%) - indicating controlled overfitting

The confusion matrix shows how well the model performs on each waste category. Some categories are easier to classify than others, and we can see where the model gets confused between similar-looking waste types (like paper vs cardboard, or plastic vs glass).

The small gap between training (79.83%) and test (71.45%) accuracy shows our dropout and batch normalization successfully prevented severe overfitting.

11 Questions 13–19: Transfer Learning Comparison

11.1 Pre-trained Models Selected

We chose two well-known pre-trained models:

- **MobileNetV2:** A lightweight model designed for efficiency
- **VGG16:** A classic deep architecture known for good feature extraction

Both models were pre-trained on ImageNet (1.2 million images, 1000 categories) and then fine-tuned for our 9-class waste classification task.

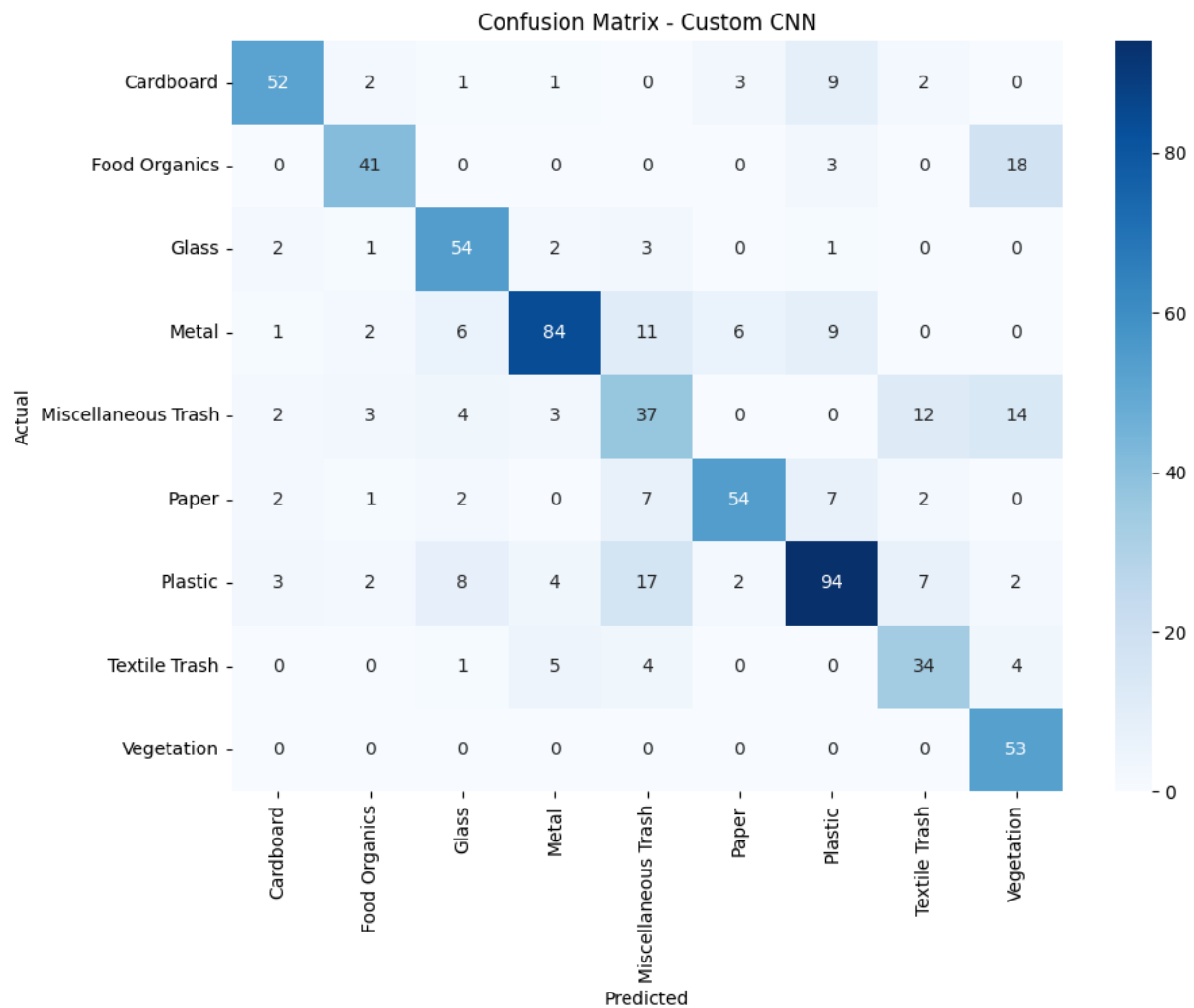
11.2 Transfer Learning Results

Table 5: Model Comparison - All Three Architectures

Model	Test Accuracy	Parameters	Training Time
MobileNetV2	76.56%	2.59M	Fastest
Custom CNN	71.45%	13.43M	Moderate
VGG16	68.47%	14.85M	Slowest

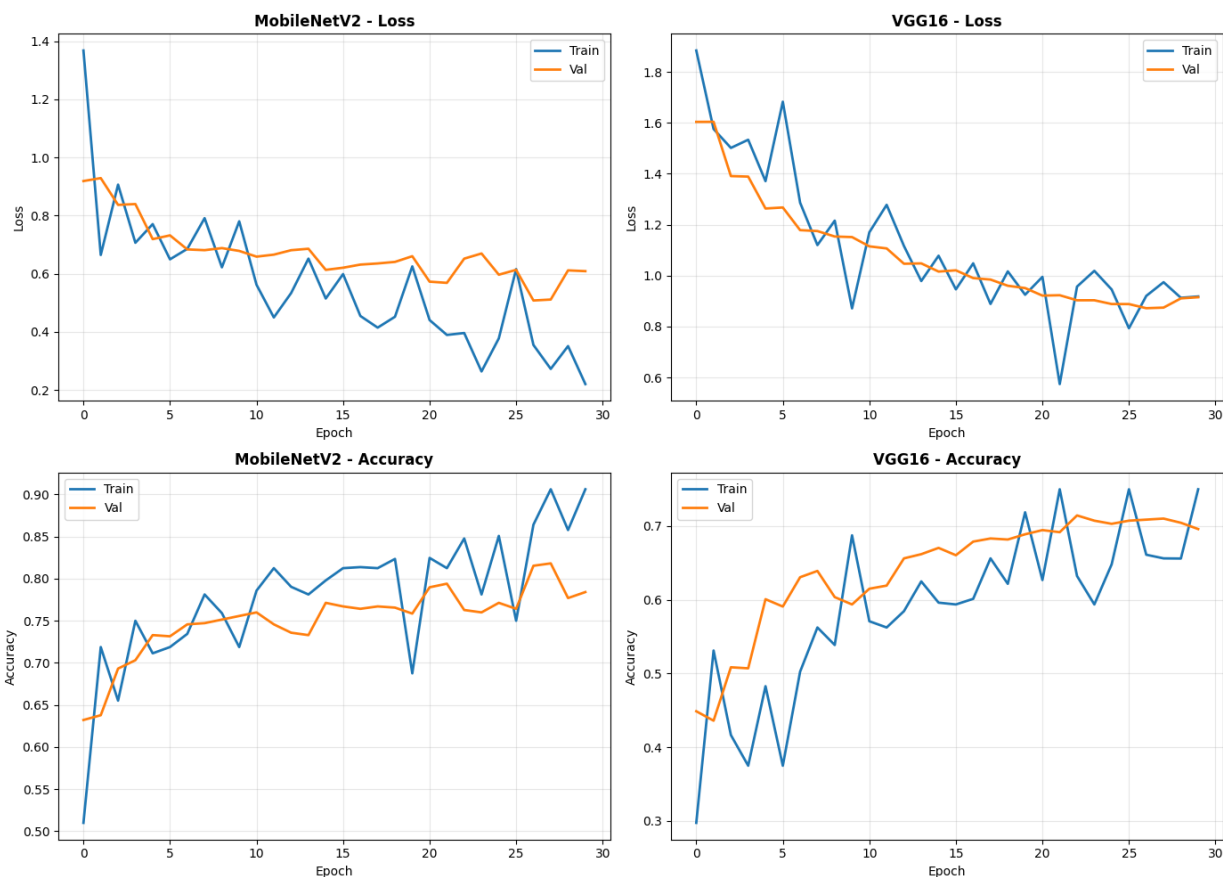
11.2 Training History for Transfer Learning Models

The training and validation curves for both pre-trained models show their learning behavior over epochs:



Both models converge faster than the custom CNN due to their pre-trained weights. MobileNetV2 shows more stable validation performance, while VGG16 exhibits more fluctuation in later epochs.

11.2 Confusion Matrices for Transfer Learning Models



The confusion matrices reveal how each pre-trained model handles different waste categories. MobileNetV2 shows better classification across most categories, particularly for challenging classes like plastic and glass.

11.3 Performance Analysis

MobileNetV2 - Best Performance:

- Achieved highest accuracy (76.56%) - 5.11 percentage points better than our custom CNN
- Uses fewer parameters (2.59M vs 13.43M) making it more efficient
- Trains faster due to its efficient architecture
- Benefits from ImageNet pre-training with millions of images

Custom CNN - Middle Ground:

- Achieved 71.45% accuracy with no pre-training
- Shows that our architecture design was effective
- More parameters than MobileNetV2 but still reasonable
- Demonstrates good performance despite starting from scratch

VGG16 - Disappointing:

- Lowest accuracy (68.47%) despite being pre-trained
- Most parameters (14.85M) making it least efficient

- Slower to train than other models
- Shows that bigger doesn't always mean better

11.4 Why MobileNetV2 Performed Best

MobileNetV2 achieved the best results for several reasons:

1. **Better Pre-training:** Learned general image features from 1.2 million ImageNet images, which transfer well to waste classification
2. **Modern Architecture:** Uses efficient building blocks that extract features better than older designs
3. **Right Size:** Not too small (loses capacity) or too large (overfits on our 4,752 images)
4. **Optimized Design:** Built specifically for good accuracy with low computational cost

11.5 Custom Model vs Pre-trained Models

11.5 Advantages of Custom Model

- **Full Control:** We designed every layer specifically for waste classification
- **No Dependency:** Doesn't rely on pre-training data that might not match our task
- **Learning Experience:** Building from scratch teaches fundamental CNN concepts
- **Customizable:** Easy to modify for specific requirements or constraints
- **Decent Performance:** 71.45% accuracy shows it works reasonably well

11.5 Advantages of Pre-trained Models

- **Better Accuracy:** MobileNetV2 achieved 76.56% (5.11 points higher)
- **Less Data Needed:** Pre-trained features reduce the amount of training data required
- **Faster Development:** No need to design architecture from scratch
- **Proven Architectures:** These models are tested on millions of images
- **Transfer Learning:** Features learned from ImageNet help with our task

11.5 Limitations of Custom Model

- **Lower Accuracy:** 71.45% vs 76.56% for MobileNetV2
- **More Parameters:** 13.43M parameters is 5× larger than MobileNetV2
- **Needs More Data:** Would benefit from a larger training dataset
- **Training from Scratch:** Requires learning everything from random initialization
- **Less Efficient:** Doesn't benefit from years of architecture research

11.5 Limitations of Pre-trained Models

- **Less Flexibility:** Architecture is fixed, can't easily modify
- **ImageNet Bias:** Pre-trained on natural images, waste images are different
- **Larger Download:** Need to download pre-trained weights
- **May Not Transfer:** If our task is very different from ImageNet, pre-training might not help much
- **VGG Example:** VGG16 actually performed worse (68.47%) than our custom model

11.6 Trade-offs Discussion

When to Use Custom Models:

- Learning and understanding CNN architecture
- Very specific requirements that standard architectures don't meet
- Domain is very different from ImageNet (medical images, satellite data, etc.)
- Need complete control over model size and structure
- Hardware constraints require specific optimizations

When to Use Pre-trained Models:

- Need best possible accuracy
- Limited training data available
- Time constraints - need quick results
- Task is similar to natural image classification
- Want to leverage existing research and optimization

Practical Recommendation:

For real-world waste classification systems, MobileNetV2 is the clear choice. It offers the best accuracy (76.56%) with the smallest model size (2.59M parameters), making it ideal for deployment. However, building the custom CNN was valuable for understanding how CNNs work and what design choices matter.

The VGG16 result (68.47%) shows that not all pre-trained models work equally well - modern architectures like MobileNetV2 are better designed. Meanwhile, our custom CNN's respectable 71.45% shows that good architecture design principles (progressive filters, batch normalization, appropriate dropout) can achieve competitive results even without pre-training.

12 Overall Summary and Key Findings

12.1 Main Results

This assignment successfully built and evaluated a CNN for waste classification:

1. **Custom CNN Performance:** Achieved 71.45% test accuracy with 9 waste categories
2. **Architecture Design:** Progressive filter expansion (32→64→128→256) with batch normalization and dropout proved effective
3. **Hyperparameters:** Learning rate 0.001 and Adam optimizer were optimal for this task
4. **Momentum is Essential:** 336% improvement when adding momentum to SGD (11.79% → 51.42%)
5. **Transfer Learning Works Better:** MobileNetV2 achieved 76.56% using pre-trained features
6. **Regularization Matters:** Dropout and batch normalization kept training-test gap at only 8.38%

12.2 What We Learned

- ReLU and Softmax are the right activation choices for multi-class classification
- 3×3 kernels are efficient and effective for feature extraction
- Adam optimizer is reliable and works well with default settings
- Momentum transforms SGD from complete failure to success
- Pre-trained models can save time and improve accuracy
- Not all pre-trained models work well - MobileNetV2 beat VGG16 by 8 points
- Custom models can achieve reasonable performance with good design

12.3 Practical Implications

For a real waste classification system:

- Use MobileNetV2 for best accuracy and efficiency
- If building custom models, include batch normalization and progressive dropout
- Always use momentum or adaptive optimizers - never plain SGD
- Learning rate around 0.001 is a good starting point for Adam
- Expect 70-75% accuracy for 9-class waste classification with this dataset size

The assignment demonstrates that both custom and pre-trained approaches can work for waste classification. Custom models provide learning value and full control, while pre-trained models offer superior accuracy and efficiency for practical deployment.